1. TRANSACTIONS & ACID PROPERTIES (DBMS + DATA ENGINEERING)

1. WHAT IS A TRANSACTION ?

- A transaction is a logical unit of work consisting of one or more database operations (read/write) that must be executed together.
- It changes the database from one consistent state to another.
- If any operation fails, the entire transaction is rolled back (no partial changes).
- Used to maintain data integrity, consistency, and handle concurrency & failures.

2. EXAMPLE - MONEY TRANSFER

Account A ₹5,000 → Transfer ₹1,000 → Account B ₹2,000

After successful transaction :

Account A	Account B
₹4,000 (- ₹1,000)	₹3,000 (+ ₹1,000)
Total = ₹ 7,000	Total = ₹ 7,000

Both debit and credit must happen together as a single transaction.

3. ACID PROPERTIES - OVERVIEW

Property	Definition	Key Points	Example
Atomicity (A)	Either all operations in the transaction commit, or none do.	• All or nothing. • If failure occurs, rollback. • Prevents partial changes.	If debit succeeds but credit fails, both are rolled back.
Consistency (C)	Transaction moves database from one valid state to another, preserving rules and constraints.	• Maintains data integrity. • Enforces constraints (primary key, foreign key, unique, check constraints). • Business rules also maintained (by application logic).	Bank balance cannot be negative. After transaction, total balance remains correct.
Isolation (I)	Concurrent transactions are controlled so they do not interfere with each other and produce incorrect results.	• Supports safe concurrency. • Different isolation levels. • Prevents anomalies like dirty read, non-repeatable read, phantom read, lost update.	Two users updating same record should not cause inconsistent results.
Durability (D)	Once a transaction is successfully committed, its changes are permanent even if there is a system crash or failure.	• Committed data is saved to disk (using techniques like Write-Ahead Logging - WAL). • Survives crashes and recovery. • Ensures persistence of committed data.	After commit, data remains even if database/server crashes.

4. TRANSACTION FLOW

1. **Begin Transaction** — `START TRANSACTION;` (or `BEGIN;`)
2. **Execute Operations** — Multiple SQL operations (INSERT/UPDATE/DELETE) as a single unit.
3. **Commit (Success)** — `COMMIT;` All changes are permanently saved.
4. **Rollback (Failure)** — `ROLLBACK;` If any operation fails, all changes are undone.

SQL EXAMPLE

```
BEGIN;
UPDATE accounts
  SET balance = balance - 1000
  WHERE account_id = 101;
UPDATE accounts
  SET balance = balance + 1000
  WHERE account_id = 102;
COMMIT;
-- If error occurs before commit
ROLLBACK;
```

5. ISOLATION LEVELS

Isolation Level	Main Idea	Prevents	Notes
Read Uncommitted	Can read uncommitted data.	-	May see dirty reads (not allowed in some DBs).
Read Committed	Reads only committed data.	Dirty Read	Default in many databases.
Repeatable Read	Repeated reads return same data.	Dirty Read, Non-repeatable Read.	Phantom reads may still occur.
Serializable	Transactions behave as if executed serially.	Dirty Read, Non-repeatable Read, Phantom Read	Strongest isolation, less concurrency, more overhead.

6. COMMON CONCURRENCY ANOMALIES (Examples)

Anomaly	What Happens ?	Example	Prevention
Dirty Read	Read uncommitted data from another transaction.	T1 updates salary to 60k (not committed). T2 reads 60k, but T1 rolls back.	Use Read Committed or higher.
Non-repeatable Read	Same query gives different results on repeated read.	T1 reads 50k, T2 updates to 60k. T1 reads again → 60k.	Use Repeatable Read or Serializable.
Phantom Read	New rows appear in a repeated query.	T1 counts 10 orders, T2 inserts new order. T1 counts again → 11.	Use Serializable.
Lost Update	Concurrent updates overwrite each other.	T1 and T2 read same balance and update. One update is lost.	Use higher isolation, locking or optimistic concurrency.

7. ACID IN DATA ENGINEERING (PRACTICAL USE CASES)

- Ensures reliable and consistent data loads in pipelines.
- Helps maintain data integrity during inserts, updates and deletes.
- Important for Slowly Changing Dimensions (SCD Type 2) to avoid partial updates.
- Used in transactional databases, Delta Lake, Iceberg, etc. (with transacbities).
- Required for idempotent and fault-tolerant data pipelines.
- Atomic table commits prevent partially written data (especially in distributed systems).
- ACID guarnatees depend on the platform (e.g., single table commit vs multi-table).

8. TRANSACTION BOUNDARIES (IMPORTANT)

- ACID guarantees apply within a transaction boundary.
- If you update two tables (orders, inventory) in a single transaction, both will commit or roll back together.
- If orders is committed seperately and then inventory fails, ACID will NOT automatically roll back the already committed orders.
- In distributed systems, cross-table transactions may not be supported (depends on platform).

9. OPTIMISTIC vs PESSIMISTIC CONCURRENCY

Optimistic Concurrency (OCC)

- Assumes conflicts are rare.
- No locking initially, checks for conflicts at commit (e.g., versioning).
- If conflict → transaction retries.
- Used in high-concurrency systems.

Example:

```
UPDATE table
SET value = 100, version = version + 1
WHERE id = 1 AND version = 5;
```

If 0 rows updated → conflict → retry.

Pessimistic Concurrency (Locking)

- Locks the data before update.
- Prevents other transactions from reading/writing (e.g., SELECT ... FOR UPDATE).
- Useful when conflicts are common.
- Can reduce concurrency.

Example:

```
SELECT * FROM inventory
WHERE product_id = 101
FOR UPDATE;
```

Locks the row until transaction completes.

10. ACID vs IDEMPOTENCY (DIFFERENCE)

ACID

- Ensures correctness of a transaction.
- All or nothing.
- Maintains consistency, isolation and durability.
- Works within transaction boundary.

Idempotency

- Ensures same operation can be executed multiple times without creating duplicate or incorrect results.
- Important for pipeline retries, reprocessed files and incremental loads.
- Required even on top of ACID.

11. SCENARIO BASED INTERVIEW QUESTIONS

1. Two tables update - one fails. What happens ?
- If both updates are in a single transaction → entire transaction rolls back (Atomicity).
- If orders is committed and inventory fails later → already committed orders will remain.
- Need to design correct transaction boundary.

2. Prevent overselling (last item in stock) ?
- Use proper transaction with isolation/locking. Atomic check & update (stock > 0).
- E.g., UPDATE inventory SET stock = stock - 1 WHERE product_id = 101 AND stock > 0;
- Handle concurrent requests and retries.

3. Job crashes after successful commit.
- Changes remain (Durability).
- If transaction committed, data survives crash.
- For Delta Lake/Iceberg, committed version is persisted even if job fails after commit.

4. Handling late-arriving data in SCD Type 2 ?
- Use transaction to expire old record & insert new record.
- Either both steps should succeed or roll back.
- Use merge/UPSERT in an atomic operation (if supported).
- Match using business key & effective dates (event time).

5. Two source systems with same customer_id ?
- Use composite business key (source_system, customer_id).
- Each source may have its own customer_id space.
- Map to unified customer dimension with surrogate key.
- Use cross-reference table for entity resolution if needed.

6. Two ETL jobs update same customer record.
- Use isolation/locking or optimistic concurrency.
- Handle retries on conflict/serialization failure.
- Ensure idempotency and deterministic logic.
- Use proper change detection & validation.

12. KEY TAKEAWAYS

- ✓ Transactions ensure reliable database operations.
- ✓ ACID guarantees: Atomicity, Consistency, Isolation, Durability.
- ✓ Isolation levels control concurrency and prevent anomalies.
- ✓ Transaction boundaries are important in multi-table updates.
- ✓ ACID and Idempotency solve different problems.
- ✓ Understand platform-specific transaction support (e.g., Delta Lake, Iceberg).
- ✓ Use proper concurrency control, retry logic and validation in pipelines.
- ✓ Common interview scenarios include failures, concurrency, SCD updates and late-arriving data.